

Number Systems and Endianness

Dr. Modhuparna Manna

Number Systems Overview

Binary (Base 2): 0,1 $\rightarrow 1011_2 = 11_{10}$

Octal (Base 8): 0–7 $\rightarrow 57_8 = 47_{10}$

Decimal (Base 10): 0–9 $\rightarrow 93_{10}$

Hexadecimal (Base 16): 0–9, A–F $\rightarrow 3A_{16} = 58_{10}$

Conversion Between Number Systems

Binary $\rightarrow$ Decimal: $1011_2 \rightarrow 11_{10}$

Decimal $\rightarrow$ Binary: $13_{10} \rightarrow 1101_2$

Binary $\rightarrow$ Octal: $110101_2 \rightarrow 65_8$

Binary $\rightarrow$ Hex: $11010111_2 \rightarrow D7_{16}$

Binary Arithmetic Operations

Addition: $101 + 11 = 1000$

Subtraction: $110 - 10 = 100$

Multiplication: $101 \times 10 = 1010$

Division: $1100 \div 10 = 110$

- Binary Addition Rules:
- $0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (carry 1)

Representing Information

- Binary: ASCII 'A' = 01000001_2
- Hexadecimal: $01000001_2 = 41_{16}$
- $A=65=41_{16}$, $B=66=42_{16}$, $C=67=43_{16}$

What is Endianness?

- Endianness = Byte order in memory
- Big-endian: MSB stored first
- Little-endian: LSB stored first

Example – 0xABBCCCDEF

- Bytes:
- AB = MSB
- BC
- CD
- EF = LSB

Big-Endian Representation

- Address | Byte
 - 0x00 → AB
 - 0x01 → BC
 - 0x02 → CD
 - 0x03 → EF
-
- Stored as: AB BC CD EF

Little-Endian Representation

- Address | Byte
 - 0x00 → EF
 - 0x01 → CD
 - 0x02 → BC
 - 0x03 → AB
-
- Stored as: EF CD BC AB

Why We Need Little Endian

1. Easier incremental access
2. Simplifies CPU arithmetic
3. Flexible data expansion
4. Hardware efficiency (Intel)
5. Forensic data interpretation

Easier Incremental Access

- In little-endian, the least significant byte (LSB) is stored at the lowest address.
- Enables fast, partial, or byte-by-byte access.
- Ideal for reading variable-length data.
- Example: 0xABBC CDEF → EF stored first, quickly accessible for 8-bit operations.
- Advantage: No rearrangement needed for smaller reads.

Simplifies CPU Arithmetic

- Arithmetic naturally starts from the least significant byte.
- CPU can perform addition/subtraction without shifting data.
- Mirrors manual arithmetic (start from rightmost digit).
- Result: Reduced instruction complexity and faster ALU operations.

Flexible Data Expansion

- Easy to extend values from 16-bit → 32-bit or 64-bit.

Example:

16-bit: 0x1234 → stored as 34 12

32-bit: 0x12345678 → stored as 34 12 56 78

- Existing bytes stay in place when widening registers.
- Advantage: Perfect for pointer casting and low-level type promotion.

Hardware Efficiency (Intel Architecture)

- Intel x86 and AMD CPUs use little-endian for simpler microarchitecture.
- Data fetch starts at lowest memory address.
- No need for internal byte reversal in pipelines.
- Improves instruction decoding and execution efficiency.
- Result: Faster and more predictable memory operations.

Forensic Data Interpretation

- Endianness determines how memory and hex data are read.
- Wrong interpretation changes values, pointers, or timestamps.
- Tools like Volatility, Autopsy, and FTK handle endianness automatically.
- Example: 32-bit timestamp EF CD BC AB – correct decoding depends on byte order.
- Essential for accurate forensic analysis and reverse engineering.

Forensics & Security Relevance of Endianness

- Memory Dumps: Endianness affects values
- Network Analysis: Big-endian standard
- Hex Editors: Show both formats

Comparison Table

Aspect	Little Endian	Big Endian
Byte Order	LSB First	MSB First
Arithmetic	Easier	Needs rearranging
Expansion	Flexible	Needs shifting
Use	Intel, ARM	Motorola, Networks